

An abstract graphic on the left side of the slide, featuring several overlapping, flowing, wavy shapes in various shades of blue, ranging from a deep navy to a bright cyan. The shapes create a sense of movement and depth.

Introduction to CRUD

CRUD Operations in Laravel

By [MOL RYNA],
[Advance Web Programming],
October 31, 2025

Introduction to CRUD

Understanding Core Data Operations

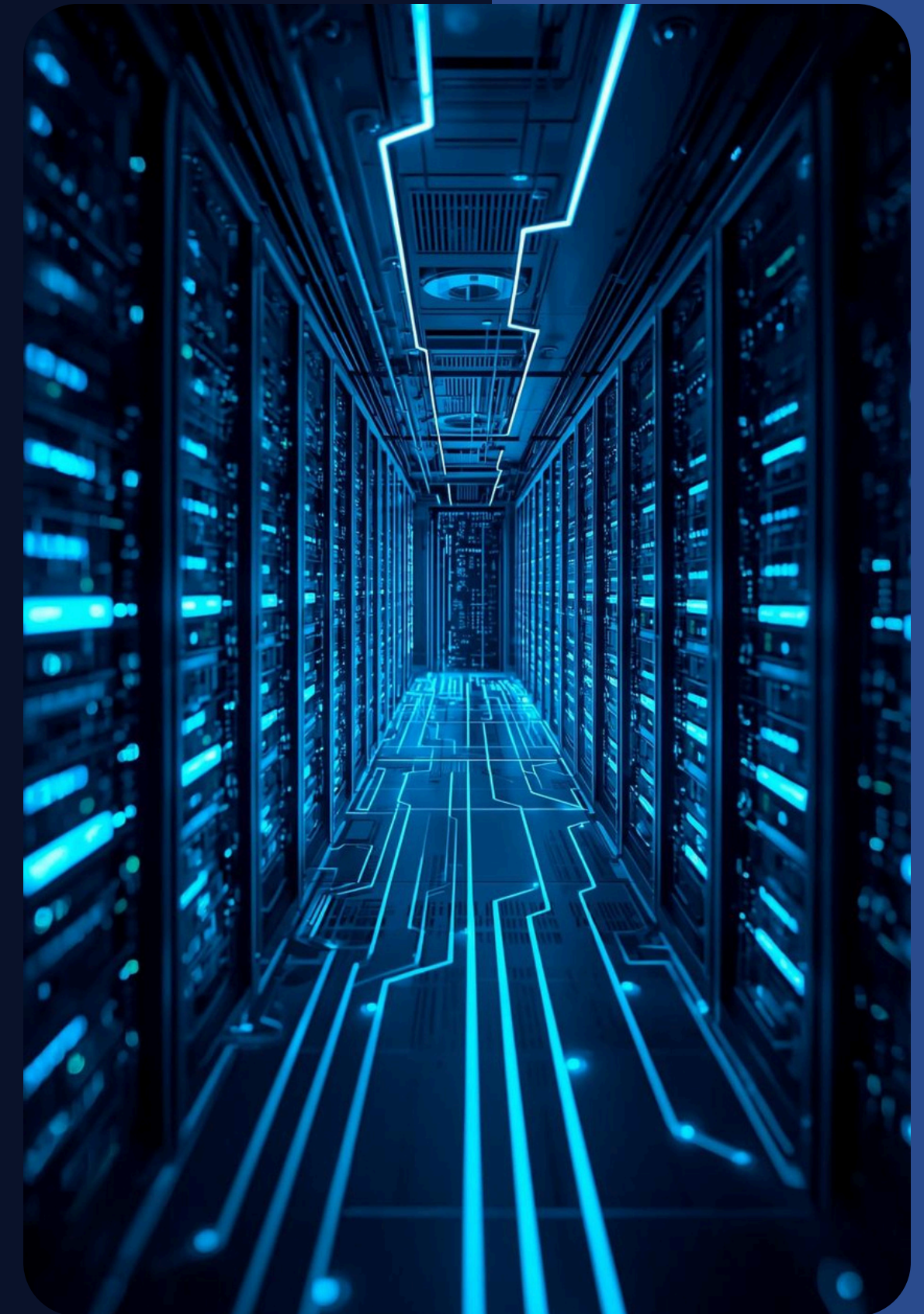
What is CRUD?

CRUD stands for **Create, Read, Update, and Delete**. These operations correspond to HTTP verbs (POST, GET, PUT/PATCH, DELETE) and are fundamental for interacting with databases in Laravel.

Why CRUD Matters

Understanding Data Flow in CRUD Operations

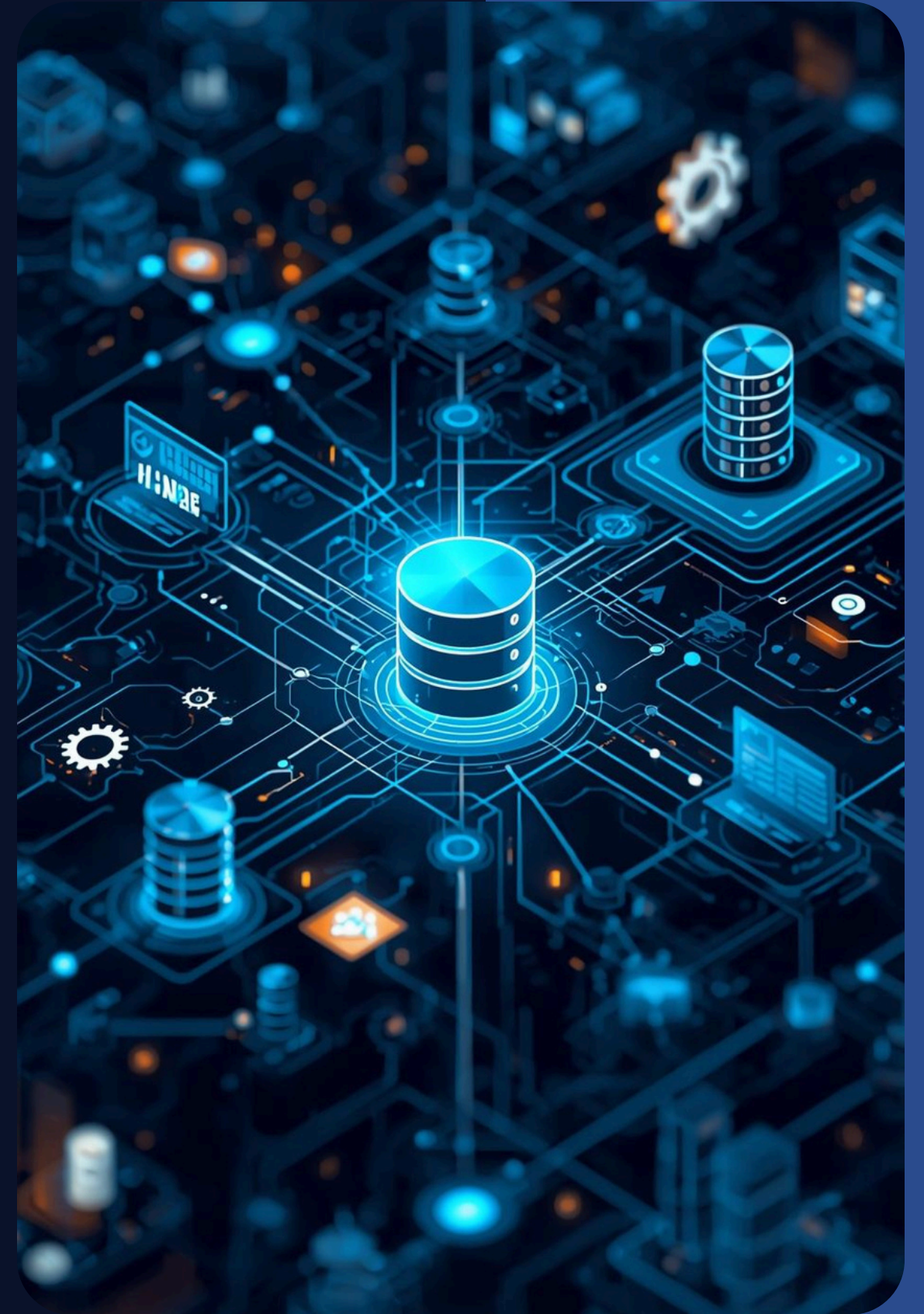
CRUD operations **enable efficient data management** between client, server, and database, ensuring seamless interactions that enhance application performance and user experience through structured processes.



Why Laravel for CRUD?

Streamlined Development with Eloquent ORM

Laravel's **Eloquent ORM** simplifies database interactions and enforces MVC architecture, ensuring structured and maintainable code. Resource routing further enhances development speed, making CRUD operations seamless and efficient.



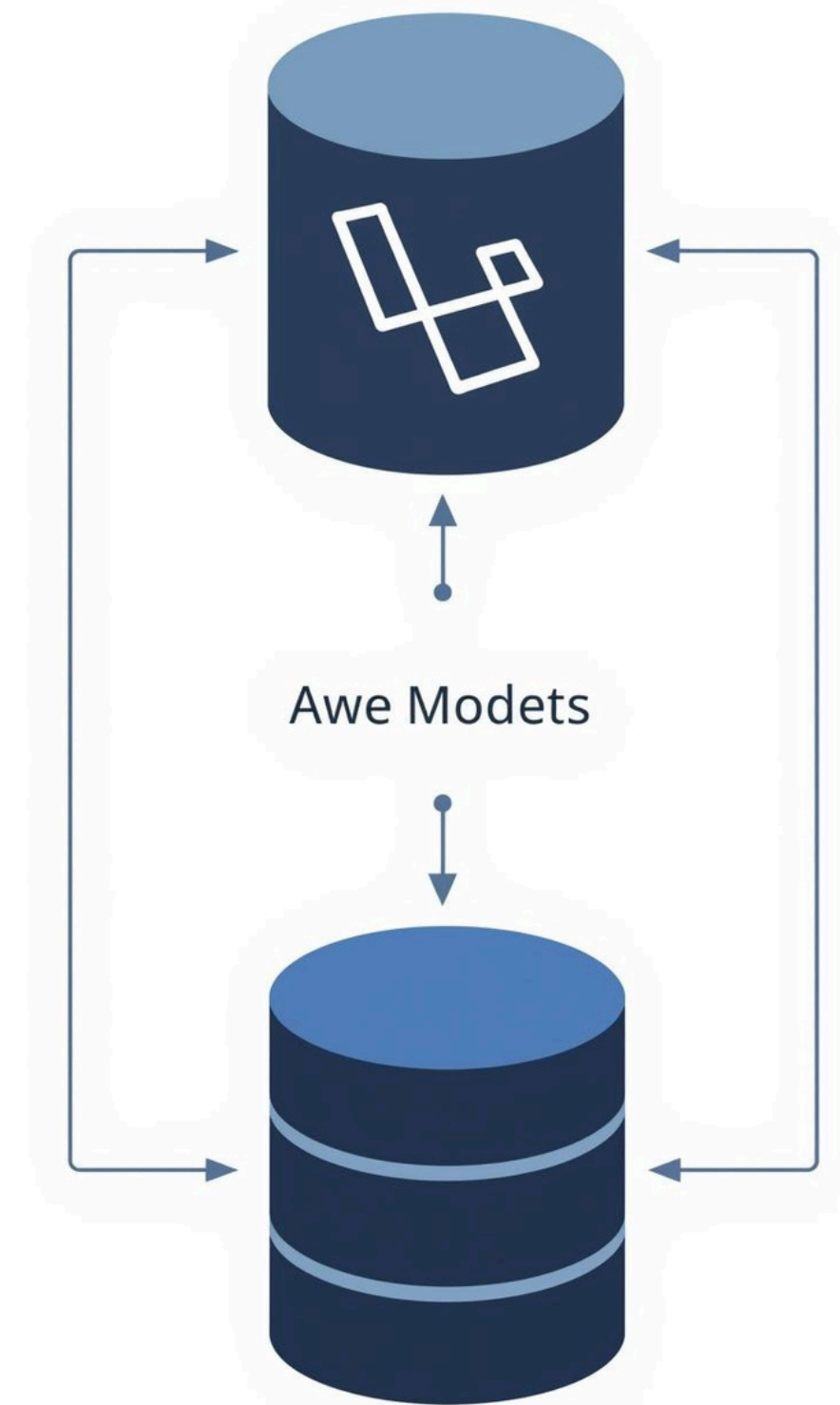
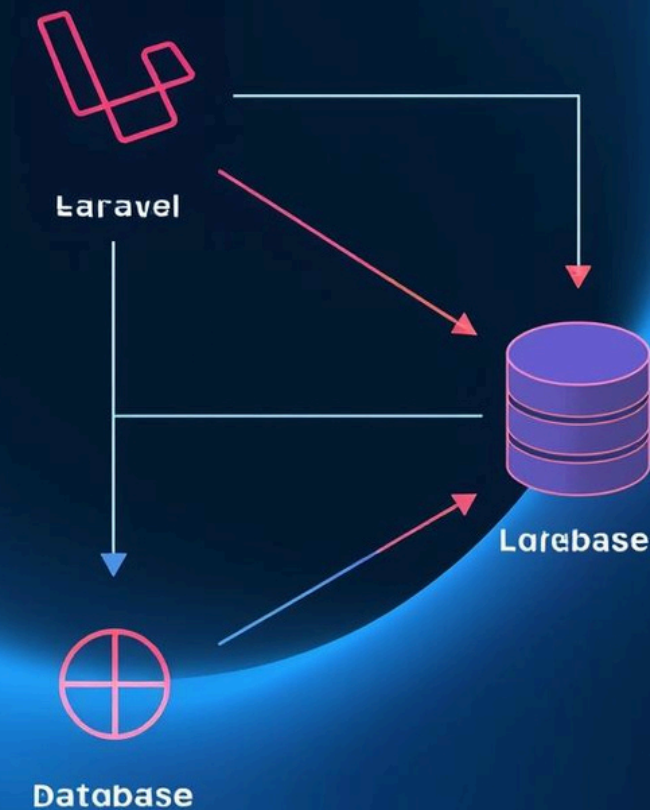
Database Migrations

Understanding Version Control in Laravel Migrations



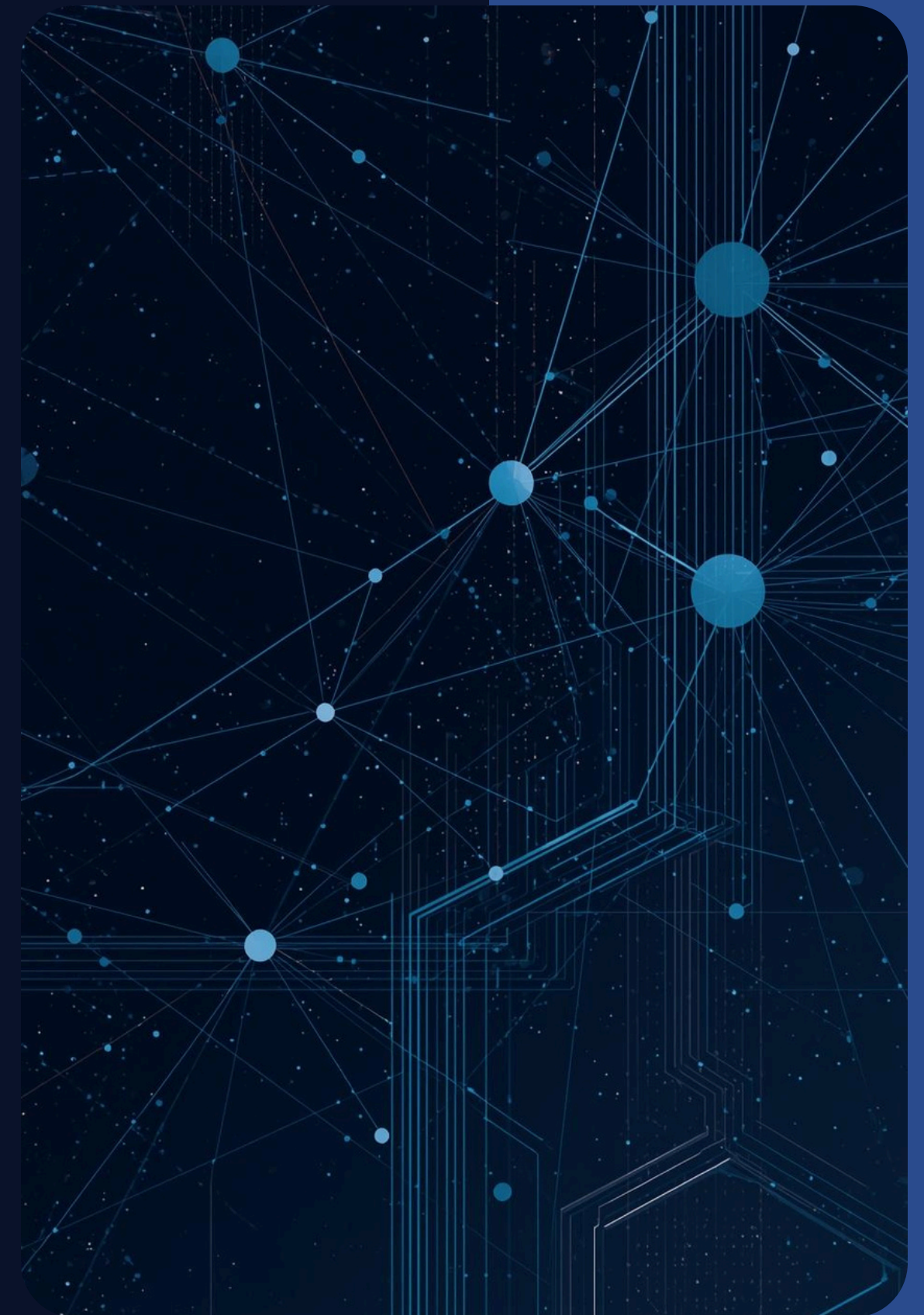
Model and Eloquent

Understanding Data Abstraction in Laravel



Routes & Controllers

Streamlined Routing with Resource Commands



Create Operation

Storing Data with Laravel

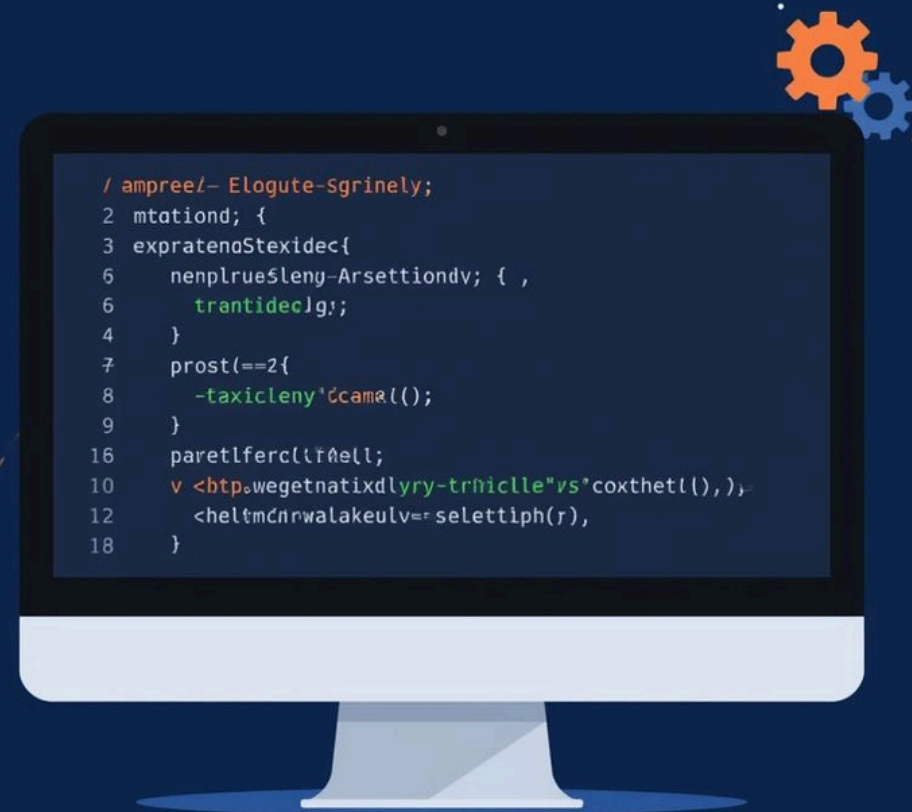
The Create operation in Laravel is essential for **storing data**. Utilizing Eloquent ORM, developers can seamlessly validate and save data through a streamlined controller method.

```
12  Hose
13  } Weter: rest(> stase Eündt/_lyan-plott):
15  } wer "(inpleante-raünel(enco))
18  4 Coner gfe lale: 0;
16  < for "opes"(ese(ttalemdav)) {
16  > chone: dop(_vefting*inoKshwent_);
17  < ather.dolest) {
18  < ("nlbate ralCimader".) {
19  fnt oBape:
10  new haspfonrtx '(cutcherröx));
19  class =nark wushofonts(("acobe""tle cr));
17  < for "ales.miste ratlvrapert" clatt):
18  > 'ouelo ftape:
17  tup-a cking tusy denexs
15  nanes
19  > think wiveryrace" reapStyvriute farehnzel); {
19  < fon: it atdloms of pice smek.e enson.
30  });
35  >>>
37  };
58  > >
```


Read Operation

Retrieving Data Efficiently

This section outlines how to efficiently retrieve data using Eloquent ORM in Laravel. It includes GET request methods, index/show functionality, and practical code examples for clarity.



The illustration shows a computer monitor with a dark blue window. The window contains a code editor with a dark background and light blue text. The code is as follows:

```

4      >
3      <S: > }
2      <S: > Becy1Dtur)
5      <N: >
7
4      <I: >: {
7
8      <S: >
10     <P: > ecdutye
17
14     <S: > {
18     <S: > eg: > {
15     <S: > >]}
10
21     <S: > }
20
>

```

The background is a light blue gradient with faint, stylized gear and circuit patterns. The monitor is a simple black rectangle with a blue glow around it.

This section covers how to efficiently **modify data** in Laravel using PUT/PATCH requests. It includes code examples for the update method, emphasizing data validation best practices for reliable operations.

Delete Operation

Understanding the DELETE Method in Laravel

The DELETE operation in Laravel allows for **safe data removal** through the destroy method. Always ensure confirmations and validations to prevent unintended data loss in applications.



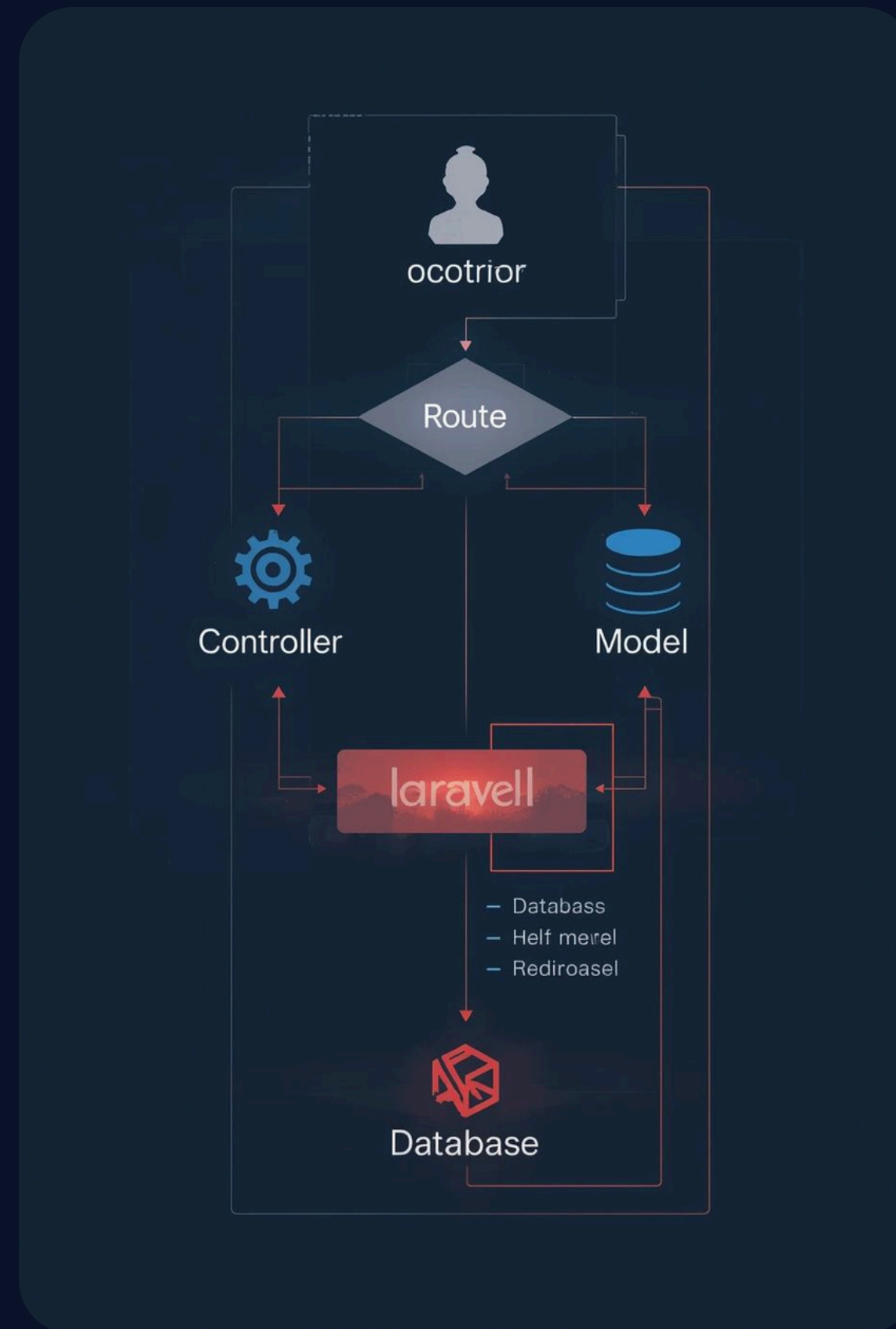
Blade Views

Displaying Data and Forms

In Laravel, Blade views facilitate dynamic content rendering. This allows developers to efficiently **display data** in tables and incorporate forms for creating or editing posts within the application.

CRUD Flow Diagram

Understanding Application Architecture in Laravel



Best Practices for CRUD

Validation

Ensure data integrity with **server-side validation** in forms.

\$fillable

Protect against mass assignment by defining **fillable attributes**.

Route Model

Simplify retrieval with **route model binding** for cleaner code.

Example Route

routes/web.php

```
Route::resource('posts', PostController::class);
```

Example Model

app/http/model/Post.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Factories\HasFactory;

class Post extends Model
{
    use HasFactory;
    protected $fillable = ['title', 'content'];
}
```

Example Controller

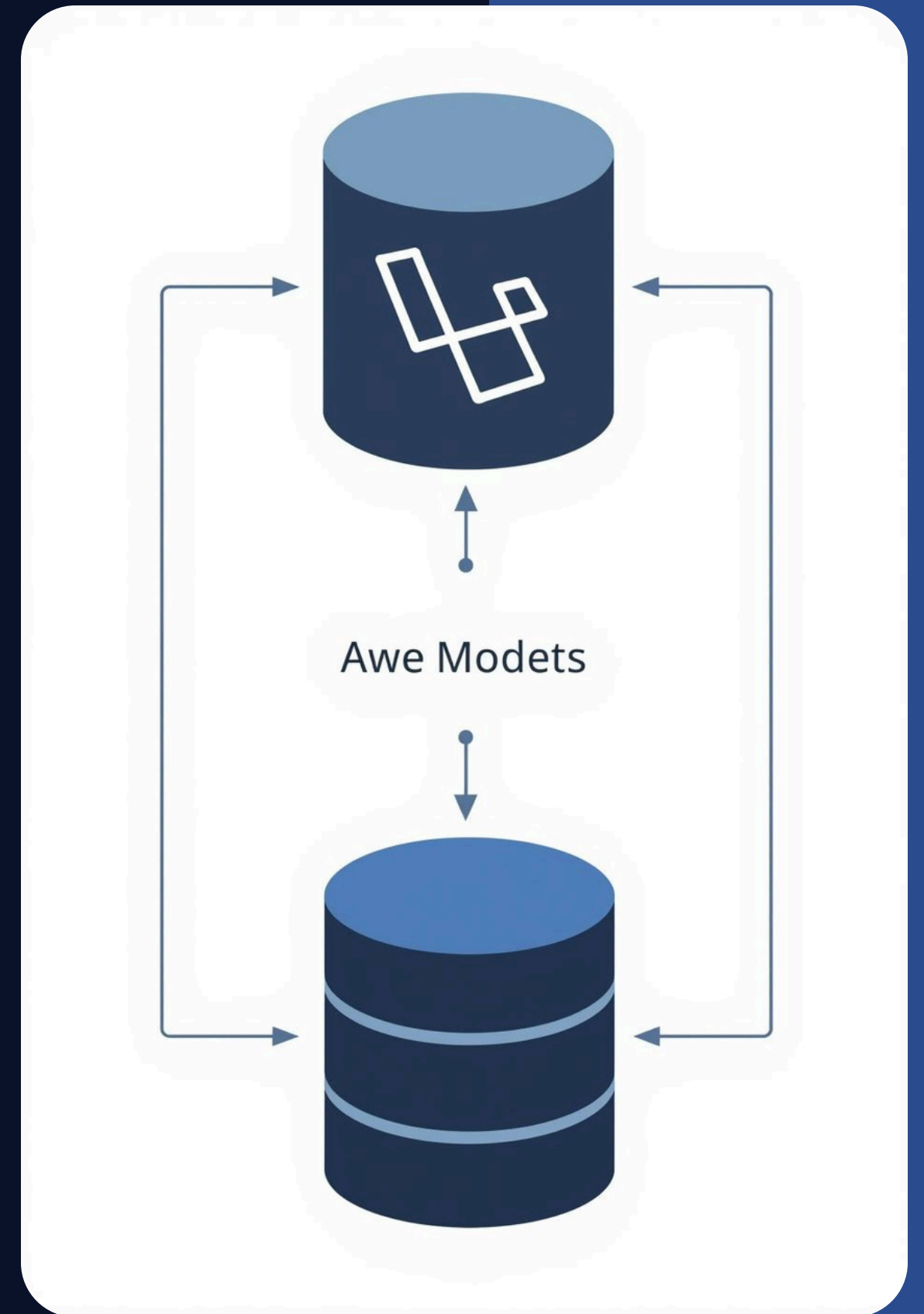
app/http/controller/PostController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Post;

class PostController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        $posts = Post::latest()->get();
        return view('posts.index', compact('posts'));
    }
}
```



Example Controller

app/http/controller/PostController.php

```
public function create()
{
    return view('posts.create');
}

/**
 * Store a newly created resource in storage.
 */
public function store(Request $request)
{
    $request->validate([
        'title' => 'required|max:255',
        'content' => 'required',
    ]);

    Post::create($request->only('title', 'content'));

    return redirect()->route('posts.index')->with('success', 'Post created successfully!');
}
```

Example Controller

app/http/controller/PostController.php

```
public function show(Post $post)
{
    return view('posts.show', compact('post'));
}

/**
 * Show the form for editing the specified resource.
 */
public function edit(Post $post)
{
    return view('posts.edit', compact('post'));
}
```

Example Controller

app/http/controller/PostController.php

```
public function update(Request $request, Post $post)
{
    $request->validate([
        'title' => 'required|max:255',
        'content' => 'required',
    ]);

    $post->update($request->only('title', 'content'));

    return redirect()->route('posts.index')->with('success', 'Post updated successfully!');
}

/**
 * Remove the specified resource from storage.
 */
public function destroy(Post $post)
{
    $post->delete();
    return redirect()->route('posts.index')->with('success', 'Post deleted successfully!');
}
}
```


Example View file index

resource/view/posts/index.blade.php

```
@extends('layouts301.app')

@section('content')
<div class="container mt-4">
    <h2>All Posts</h2>

    @if(session('success'))
        <div class="alert alert-success">{{ session('success') }}</div>
    @endif

    <a href="{{ route('posts.create') }}" class="btn btn-primary mb-3">+ Create New Post</a>

    <table class="table table-bordered">
        <thead>
            <tr>
                <th>Title</th>
                <th>Content</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody>
            @foreach($posts as $post)
                <tr>
                    <td>{{ $post->title }}</td>
                    <td>{{ $post->content }}</td>
                    <td>
                        <a href="{{ route('posts.show', $post) }}" class="btn btn-info btn-sm">View</a>
                        <a href="{{ route('posts.edit', $post) }}" class="btn btn-warning btn-sm">Edit</a>
                        <form action="{{ route('posts.destroy', $post) }}" method="POST" style="display:inline-block">
                            @csrf @method('DELETE')
                            <button class="btn btn-danger btn-sm" onclick="return confirm('Delete this post?')">Delete</button>
                        </form>
                    </td>
                </tr>
            @endforeach
        </tbody>
    </table>
</div>
@endsection
```

Example View file create

resource/view/posts/create.blade.php

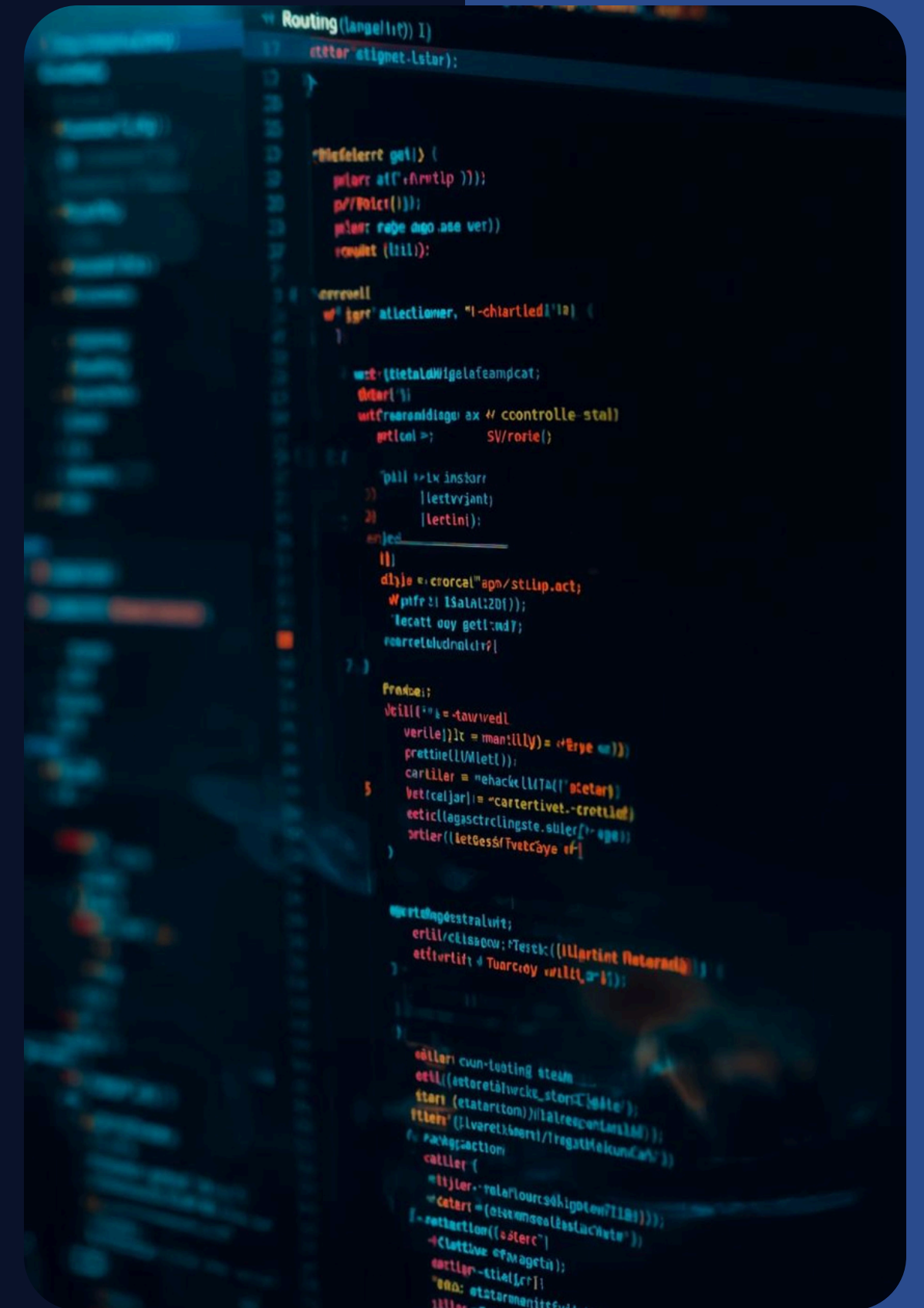
```
@extends('layouts301.app')

@section('content')
<div class="container mt-4">
    <h2>Create New Post</h2>

    <form action="{{ route('posts.store') }}" method="POST">
        @csrf
        <div class="mb-3">
            <label>Title</label>
            <input type="text" name="title" class="form-control" value="{{ old('title') }}">
            @error('title') <span class="text-danger">{{ $message }}</span> @enderror
        </div>

        <div class="mb-3">
            <label>Content</label>
            <textarea name="content" class="form-control">{{ old('content') }}</textarea>
            @error('content') <span class="text-danger">{{ $message }}</span> @enderror
        </div>

        <button type="submit" class="btn btn-success">Save</button>
        <a href="{{ route('posts.index') }}" class="btn btn-secondary">Back</a>
    </form>
</div>
@endsection
```



Example View file edit

resource/view/posts/edit.blade.php

```
@extends('layouts301.app')

@section('content')
<div class="container mt-4">
    <h2>Edit Post</h2>

    <form action="{{ route('posts.update', $post) }}" method="POST">
        @csrf @method('PUT')

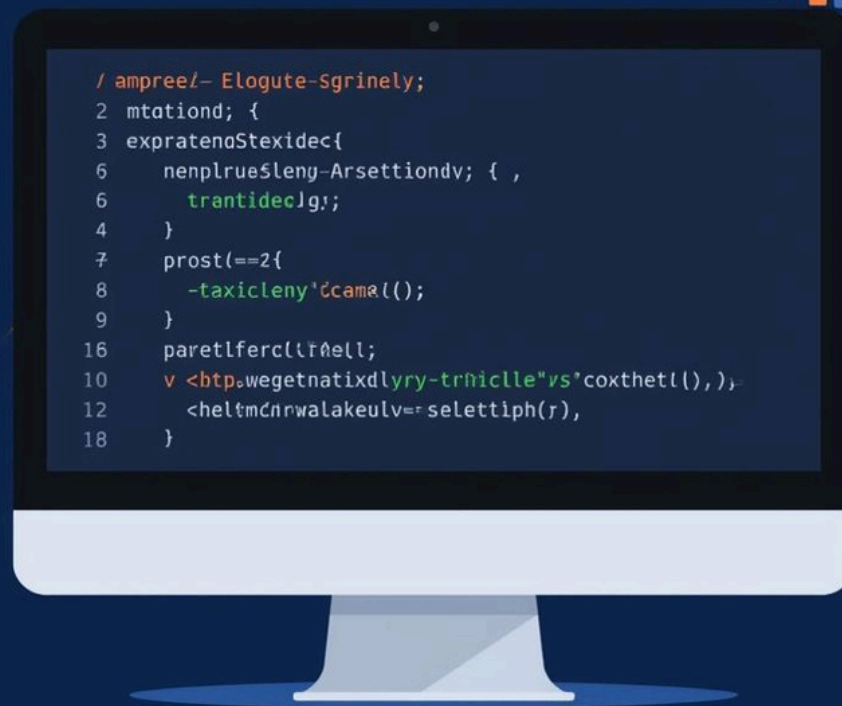
        <div class="mb-3">
            <label>Title</label>
            <input type="text" name="title" class="form-control" value="{{ old('title', $post->title) }}">
        </div>

        <div class="mb-3">
            <label>Content</label>
            <textarea name="content" class="form-control">{{ old('content', $post->content) }}</textarea>
        </div>

        <button type="submit" class="btn btn-primary">Update</button>
        <a href="{{ route('posts.index') }}" class="btn btn-secondary">Cancel</a>
    </form>
</div>
@endsection
```


Example View file show

resource/view/posts/show.blade.php



```
@extends('layouts301.app')

@section('content')
<div class="container mt-4">
    <h2>{{ $post->title }}</h2>
    <p>{{ $post->content }}</p>
    <a href="{{ route('posts.index') }}" class="btn btn-secondary">Back</a>
</div>
@endsection
```

Tools and Packages

Enhancing Developer Productivity with Laravel

Have any Question ?



Contact Information

Get in Touch with Us

Phone

096 98 94 789

Email

Rynaboy22@gmail.com

Thank you see you next weeks